1.- CAPÍTULO II: Sentencias de Selección

Introducción a Python

Marzo 2026 - Agosto 2026

Ciencia de Datos & Inteligencia Artificial

Contenido de la presentación

1. Introducción a Python

1.1 Introducción
1.2 IDE
1.3 Tipos de datos en Python
1.4 Errores de tecleo y excepciones
1.5 Entrada y Salida de datos
1.6 Ejemplos
1.7 Ejercicios Varios

Contenido de la presentación

1. Introducción a Python

1.1 Introducción

1.2 IDE

1.3 Tipos de datos en Python

1.4 Errores de tecleo y excepciones

1.5 Entrada y Salida de datos

1.6 Ejemplos

1.7 Ejercicios Varios

1.1. Introducción

- Python es un lenguaje de programación creado por Guido van Rossum a principios de los años 90 cuyo nombre está inspirado en el grupo de cómicos ingleses “Monty Python”.
- Es un lenguaje similar a Perl, pero con una sintaxis muy limpia y que favorece un código legible.
- Python tiene, no obstante, muchas de las características de los lenguajes compilados, por lo que se podría decir que es semi interpretado.
- En Python, como en Java y muchos otros lenguajes, el código fuente se traduce a un pseudo código máquina intermedio llamado bytecode la primera vez que se ejecuta, generando archivos .pyc o .pyo (bytecode optimizado), que son los que se ejecutarán en sucesivas ocasiones.

1.1. Tipado dinámico

- Se refiere a que no es necesario declarar el tipo de dato que va a contener una determinada variable, sino que su tipo se determinará en tiempo de ejecución según el tipo del valor al que se asigne.
- El tipo de esta variable puede cambiar si se le asigna un valor de otro tipo.

1.1. Fuertemente tipado

El intérprete de Python está disponible en multitud de plataformas (UNIX, Solaris, Linux, DOS, Windows, OS/2, Mac OS, etc.).

1.1. Orientado a objetos

- La orientación a objetos es un paradigma de programación en el que los conceptos del mundo real relevantes para nuestro problema se tras- ladan a clases y objetos en nuestro programa.
- Python también permite la programación imperativa, programación funcional y programación orientada a aspectos.

1.1. Por qué Python

- Su sintaxis simple, clara y sencilla; el tipado dinámico, el gestor de memoria, la gran cantidad de librerías disponibles y la potencia del lenguaje.
- Python también permite la programación imperativa, programación funcional y programación orientada a aspectos.

Contenido de la presentación

1. Introducción a Python

1.1 Introducción

1.2 IDE

1.3 Tipos de datos en Python

1.4 Errores de tecleo y excepciones

1.5 Entrada y Salida de datos

1.6 Ejemplos

1.7 Ejercicios Varios

1.2. IDE

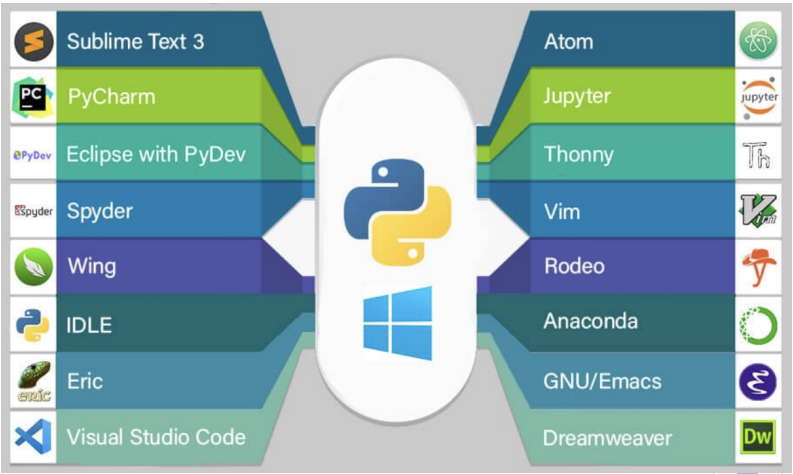
- Un IDE (Entorno de Desarrollo Integrado, por sus siglas en inglés) es una aplicación de software que combina todas las herramientas esenciales para programar en un solo lugar, maximizando la productividad.
- Incluye un editor de código, depurador (debugger), compilador/intérprete y automatización de tareas en una sola interfaz gráfica.

1.2. IDE

- Editor de Texto: Permite escribir el código fuente, usualmente con resaltado de sintaxis.
- Compilador/Intérprete: Traduce el código a lenguaje de máquina para ejecutarlo.
- Depurador (Debugger): Ayuda a localizar y corregir errores en el código.
- Automatización: Facilita compilar, probar y empaquetar el software.

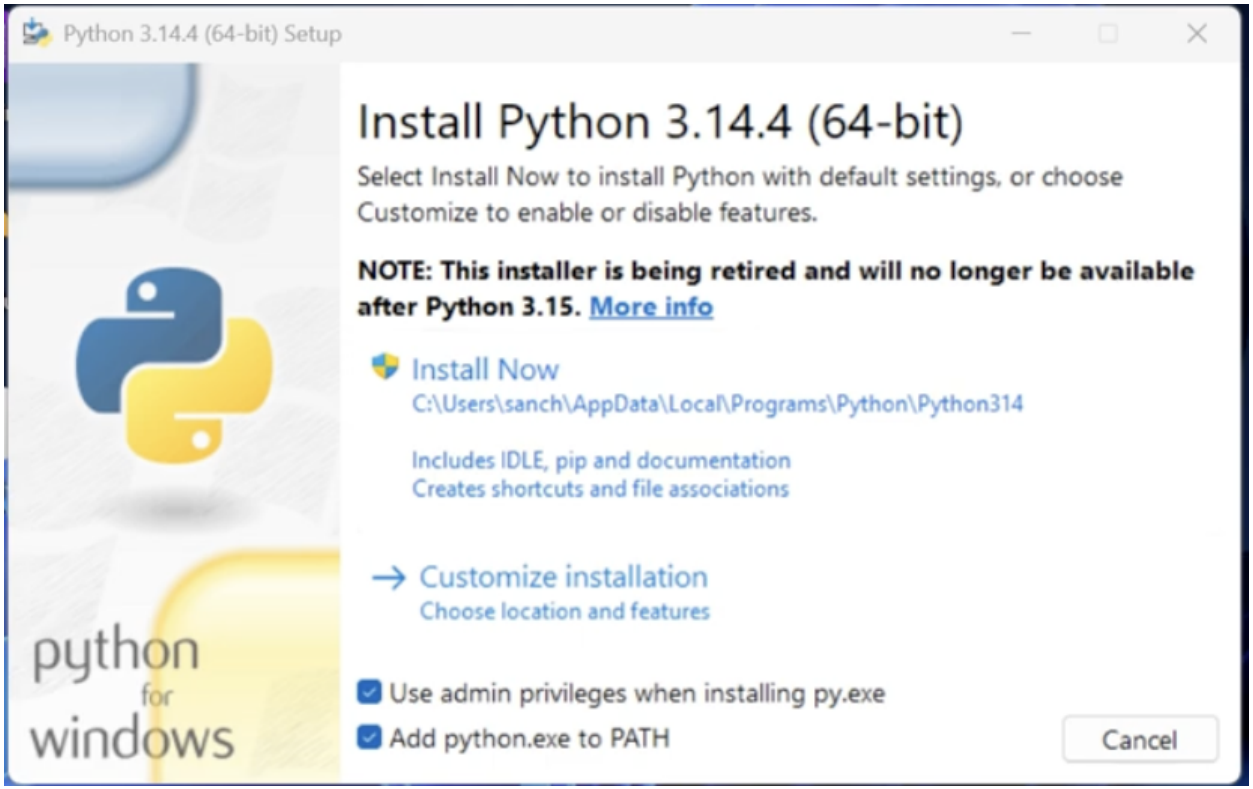
1.2. IDE Python

- PyCharm: Ampliamente considerado el IDE dedicado a Python más potente. Ofrece una edición Community (gratuita y de código abierto).
- Visual Studio Code (VS Code): Un editor ligero y altamente extensible que se convierte en un IDE completo mediante la extensión de Python. Es preferido por su versatilidad para trabajar con múltiples lenguajes.
- Jupyter Notebook / Lab: Un entorno interactivo basado en la web, ideal para análisis de datos, visualización y aprendizaje automático.



1.2. Instalación de Python

- Descargar Python: `https://www.python.org/downloads/`
- Tutorial `https://www.youtube.com/watch?v=E9Eu3fGvwa0`



1.2. Instalación de PyCharm

- Descargar PyCharm: `https://www.jetbrains.com/pycharm/download/`
- Tutorial `https://www.youtube.com/watch?v=dJjPq2xlLcQ`

Contenido de la presentación

1. Introducción a Python

1.1 Introducción

1.2 IDE

1.3 Tipos de datos en Python

1.4 Errores de tecleo y excepciones

1.5 Entrada y Salida de datos

1.6 Ejemplos

1.7 Ejercicios Varios

1.3. Enteros & Reales & Booleanos

- Los números enteros son aquellos números positivos o negativos que no tienen decimales.
- Python puede almacenar números enteros de -2^{31} a $2^{31}-1$, o lo que es lo mismo, de −2.147.483.648 a 2.147.483.647. En plataformas de 64 bits, el rango es de −9.223.372.036.854.775.808 hasta 9.223.372.036.854.775.807.
- Los números reales son aquellos números positivos o negativos que tienen decimales.
- Python puede representar valores que van desde $2,2250738585072020x10^{-308}$ hasta $1,79769313486231571 0308$.
- Números más grandes -> se convierten en infinito.
- Números más pequeños (cercanos a 0) -> pueden convertirse en 0.
- Tipos booleanos: True o False, es decir, verdadero o falso.
- Operadores booleanos: «y lógica» o conjunción (and), la «o lógica» o disyunción (or) y el «no lógico» o negación (not).

1.3. Enteros & Reales

- entero = 23 # Entero
- real = 0.1e-3 # Real
- real = 0.1e+4 # Real

Contenido de la presentación

1. Introducción a Python

1.1 Introducción

1.2 IDE

1.3 Tipos de datos en Python

1.4 Errores de tecleo y excepciones

1.5 Entrada y Salida de datos

1.6 Ejemplos

1.7 Ejercicios Varios

1.4. Excepciones

Cuando introducimos una expresión y damos la orden de evaluarla, es posible que nos equivoquemos.

- Si hemos formado incorrectamente una expresión, Python nos lo indicará con un mensaje de error.
- El mensaje de error proporciona información acerca del tipo de error cometido y del lugar en el que este ha sido detectado.
- En Python los errores se denominan excepciones. Cuando Python es incapaz de analizar una expresión, produce una excepción.
- $1 + 2)$
- $1 + *8$
- $1 + 2\,\%$
- 12
- $mm + 2$

Contenido de la presentación

1. Introducción a Python

1.1 Introducción
1.2 IDE
1.3 Tipos de datos en Python
1.4 Errores de tecleo y excepciones
1.5 Entrada y Salida de datos
1.6 Ejemplos
1.7 Ejercicios Varios

1.5. Entrada y Salida de datos

Para recibir (leer) y escribir (mostrar) valores en Python, se usan principalmente *input*() y *print*().

- nombre = input('Ingresa tu nombre: ')
- edad = int(input('Ingresa tu edad: '))
- altura = float(input('Ingresa tu altura: '))

- print('Hola', nombre)
- print('Tu edad es:', edad)

- print(f'Hola {nombre}, tienes {edad} años')

1.5. Entrada y Salida de datos

IBARRA - ECUADOR

- text1 = ' Hello '
- text2 = ' world '
- var_num = 1
- print (text1 , text2 , var_num)

- num = 2
- print ('El número es : ' + num)
- print ('El número es : ' + str(num))
- print ('El número es : ' , num)
- print (f'El número es : {num}')

- print(f'Hola {nombre}, tienes {edad} años')

1.5. Entrada y Salida de datos

IBARRA - ECUADOR

- nombre = input ('Diga un nombre : ')
- num1 = int (input (' Ingrese un número : '))
- num2 = int (input (' Ingrese otro número : '))
- print ('La suma de los números de ' + nombre + ' es : ' + str (num1 + num2))

- num = 2
- print ('El número es : ' + num)
- print ('El número es : ' + str(num))
- print ('El número es : ' , num)
- print (f'El número es : {num}')

- print(f'Hola {nombre}, tienes {edad} años')

Contenido de la presentación

1. Introducción a Python

1.1 Introducción
1.2 IDE
1.3 Tipos de datos en Python
1.4 Errores de tecleo y excepciones
1.5 Entrada y Salida de datos
1.6 Ejemplos
1.7 Ejercicios Varios

1.6. Ejemplos

- $4/2$
- $3/2$
- $4//2$
- $3//2$
- $2+3$
- $2.0+3$
- $2+3.0$
- $2.0+3.0$
- $1+2+3+4+5+0.5$
- $1+2+3+4+5+0.5$

1.6. Ejemplos

- True and False
- not True
- (True and False) or True
- True and True or True
- 2 == 3
- 2 == 2
- 3.1 == 3.1
- True == True
- True == False
- 2 == 1 + 1
- (True or (2 == 1 + 1)) == True

1.6. Ejemplos

- $2 < 1$
- $1 < 2$
- $5 > 1$
- $5 > 5$
- $5 >= 5$
- $2 != 2$
- $3.1 != 3.5$
- True == True
- True != False
- $2 != 1 + 1$
- (True or $(2 == 1 + 1)$) != True

1.6. Ejemplos: Variables y asignaciones

- pi = 3.141592
- r = 5
- perimetro = 2 * pi * r
- area = pi * r ** 2
- print(perimetro)
- print(area)

1.6. Ejemplos

- a = 1
- 2 * a
- a + 2
- a = 2
- a * a
- a = 5
- a == 5

NOTA: El nombre de una variable es su identificador.

1.6. Ejemplos

- dos palabras = 1
- _ = 1
- 12hora = 1
- hora12 = 1
- año = 1
- from = 1
- var! = 1
- 'var' = 1
- a(b) = 1
- 12 = 1
- uno.dos = 1
- área = 1
- area = 1
- area-rect = 1

1.6. Ejemplos

- x__1 = 1
- __1 = 1
- _x_ = 1
- x_x = 1
- X_X = 1

1.6. Ejemplos: String

- nombre = 'Juan'
- print (nombre)
- nombre2 = 'Juan'
- print (nombre2)

1.6. Ejemplos: String

- r = 'a' + 'b'
- print (r)
- nombre = 'Juan'
- r = nombre + 'Cano'
- print (r)

- nombre = 'Juan'
- apellido = 'Cano'
- r = nombre + apellido
- print (r)

- nombre = Juan

- Juan = 'Juan'
- nombre = Juan
- print(nombre)

1.6. Ejemplos: String

- '12' + '12'
- 12 + 12
- 12 + '12'

- 'Hola' * 5
- '-' * 10

1.6. Ejemplos: String

- a = 'b'
- a + 'b'
- a + 'a'
- a * 2 + 'b' * 3
- 2 * (a +'b')
- 2 * ('a' +'b')

1.6. Ejemplos: String

- 'a' * 3 + '/*' * 5 + 2 * 'abc' + '+'
- palabra = 'abcba'
- (4 * '<' + palabra + '>' * 4) * 2
- subcadena = '=' + '-' * 3 + '='
- '10' * 5 + 4 * subcadena
- 2 * '12' + '.' + '3' * 3 + 'e-' + 4 * '76'

Contenido de la presentación

1. Introducción a Python

1.1 Introducción
1.2 IDE
1.3 Tipos de datos en Python
1.4 Errores de tecleo y excepciones
1.5 Entrada y Salida de datos
1.6 Ejemplos
1.7 Ejercicios Varios

1.7. Ejercicios

Resolver los ejercicios planteados en los Diagramas de Flujo.